

AgriSenseAI Functional Documentation

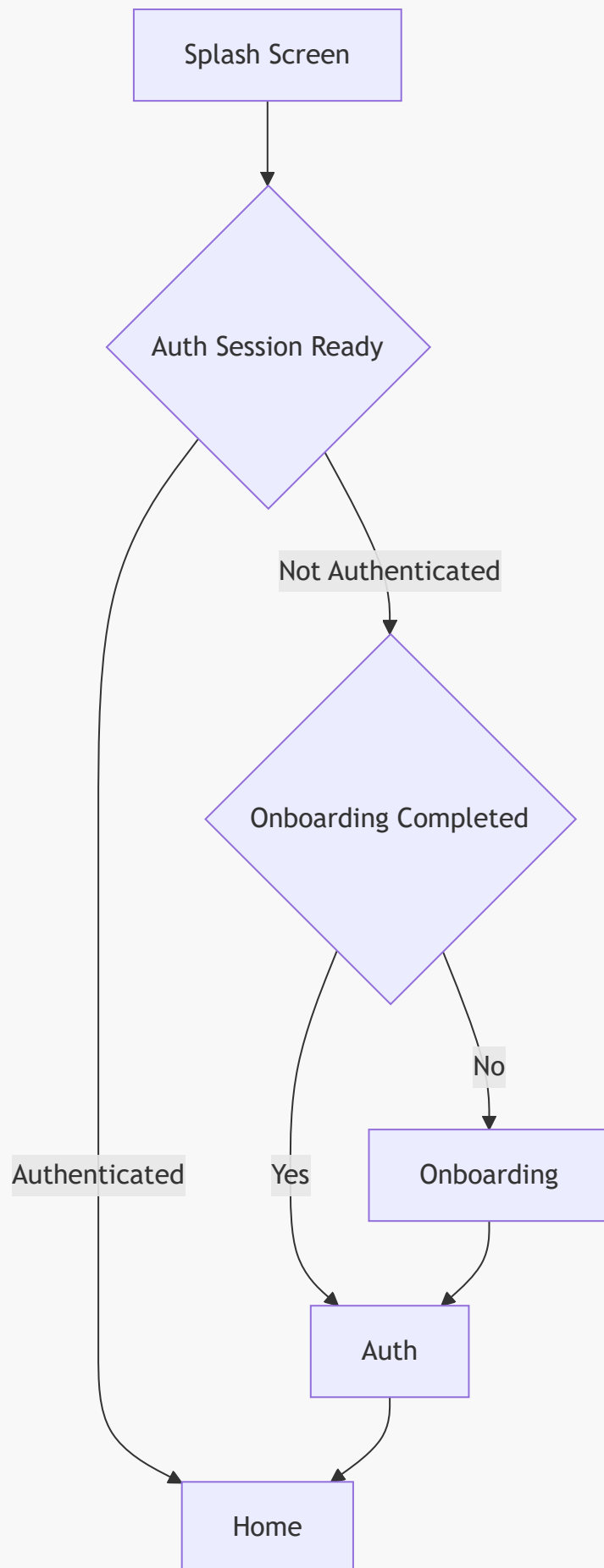
Purpose

This document describes how the current project works, how data is displayed, the app navigation flow, Firebase configuration, and the planned sensor connectivity flow (Bluetooth and Wi-Fi). It is based on the current codebase and highlights what is implemented vs. planned.

App Flow (Implemented)

- Router and auth redirects are defined in [lib/core/router/app_router.dart](#).
- Initial boot starts at Splash in [lib/core/router/route_names.dart](#).
- App entry initializes Firebase, DI, and responsive config in [lib/main.dart](#).

Mermaid flow (use this in a PDF export):



Notes:

- Splash navigation decisions are in [lib/features/splash_onboarding/presentation/screens/splash_screen.dart](#).
- Onboarding completion is stored using secure storage in [lib/core/services/local_storage/local_storage_service.dart](#).

How Sensor Data Is Shown (Current Implementation)

- Sensor cards are rendered in [lib/features/home/presentation/widgets/home_dashboard_widgets.dart](#).
- Data displayed on the home dashboard comes from a local mock data source in [lib/features/home/data/datasources/home_local_datasource.dart](#).
- Sensor models and entities are defined in [lib/features/home/data/models/sensor_reading_model.dart](#) and [lib/features/home/domain/entities/sensor_reading.dart](#).

Current flow for data display:

1. Home screen calls `loadDashboard()` in [lib/features/home/presentation/providers/home_provider.dart](#).
2. Provider fetches mock dashboard data from [lib/features/home/data/datasources/home_local_datasource.dart](#).
3. UI renders `HomeSensorCard` widgets for each `SensorReading` in [lib/features/home/presentation/widgets/home_dashboard_widgets.dart](#).

Sensor Connectivity (Bluetooth and Wi-Fi)

Current status

- There is no production Bluetooth/Wi-Fi connection service in the codebase yet.
- The Devices feature is currently driven by mock data in [lib/features/devices/data/datasources/devices_local_datasource.dart](#).

Proposed implementation plan (to make connectivity functional)

This section is a plan, not yet implemented.

1. Bluetooth discovery and pairing

- Create a core service (e.g., [core/services/bluetooth/bluetooth_service.dart](#)) that wraps a Bluetooth package.
- Expose: `scan()`, `connect(deviceId)`, `disconnect()`, and `streamConnectionState()`.
- Store paired devices locally (secure storage or a small local DB).

2. Wi-Fi provisioning

- Create a core service (e.g., [core/services/wifi/wifi_provisioning_service.dart](#)).
- Implement device provisioning flow: connect to device AP, send Wi-Fi SSID/password, then verify device online.

3. Sensor data ingestion

- Add a data source that reads device data either from:
 - Bluetooth GATT characteristics, or
 - Wi-Fi (HTTP or MQTT) to a device gateway.

- Map raw data into `SensorReading` models.

4. Repository layer wiring

- Add a device repository for fetching live sensor data.
- Swap home dashboard local datasource for real data source once stable.

5. UI updates

- Update devices list to display real connection state.
- Update home dashboard to show live sensor stream and last update times.

Firestore Setup (Implemented + Required Steps)

Implemented in code

- Firestore options are generated in `lib/firestore_options.dart`.
- Firestore initialization is in `lib/main.dart`.
- Project metadata is stored in `firebase.json`.

Required setup steps (Firestore Console)

1. Create a Firestore project (already set to `agrisenseai-app`).
2. Add Android and iOS apps:
 - Android application id and config are referenced in `firebase.json`.
 - Download `google-services.json` and place it at `android/app/google-services.json`.
 - For iOS, add `GoogleService-Info.plist` to the iOS Runner target.
3. Enable Authentication:
 - Enable Google provider if using Google Sign-In.
4. Configure Cloud Messaging (Notifications):
 - Ensure FCM is enabled.
 - Verify notification channels in `lib/core/services/notifications/notification_local_handler.dart`.
 - Background handler is defined in `lib/core/services/notifications/notification_background_handler.dart`.

Note: `NotificationService` is currently a placeholder in `lib/core/services/notifications/notification_service.dart`.

Theme Reference (Use in PDF)

Theme tokens are defined in `lib/core/theme`:

- Colors: `lib/core/theme/app_colors.dart`
- Typography: `lib/core/theme/app_text_style.dart`
- Fonts: `lib/core/theme/app_fonts.dart`
- Spacing: `lib/core/theme/app_spacing.dart`
- Border Radius: `lib/core/theme/app_border_radius.dart`

Recommended summary block for the PDF:

- Primary color: `#2E7D32`

- Accent blue: #4FC3F7
- Accent yellow: #FBC02D
- Background: #F5F7F4
- Surface: #FFFFFF
- Font family: Poppins (Regular/Medium/SemiBold)
- Rounded corners: 16px buttons, 24px cards

Next Steps to Fully Functional Hardware Integration

- Implement Bluetooth and Wi-Fi services in core.
- Add repository + datasource for live sensor data.
- Replace home local datasource with live datasource.
- Add device provisioning UI and error states.

If you want, I can expand this into a full PDF-ready document with diagrams for devices, auth, and data pipelines once the connectivity services are implemented.